Review vulnerable login logic in lab/app py

ID: local-5adc3cc82cef1c653062bafe2b30add1658daecd24cbb604f39c4ac3ae8df7b5

Model: upstage/solar-mini4:free **Started:** 2026-11-08 21:19:11

System Prompt (Persona)

USER 2026-11-08 21:20:19

Como especialista em AppSec, revise lab/app.py. É um laboratório próprio, sem servidor, com SQLite em memória e credenciais fictícias. Aponte a linha vulnerável, explique como a entrada pode contornar o login e mostre como reescrever a função com uma consulta parametrizada. Explique em que cenário o MITRE ATT&CK T1190 se aplicaria. Faça a revisão sem editar arquivos.

ASSISTANT 2026-11-08 21:20:28

TOOL 2026-11-08 21:20:35

ASSISTANT 2026-11-08 21:20:52

Revisão — `lab/app.py`

Linha vulnerável: 17

```python
consulta = f"SELECT 1 FROM usuarios WHERE nome = '{nome}' AND
```

senha = '{senha}'"
```

### Como o login pode ser contornado

Os valores de `nome` e `senha` são injetados **diretamente** na string SQL, sem sanitização e sem parâmetros. A entrada não é tratada como dado — o SQLite a interpreta como parte da query.

**Cenário 1 — burlar só a senha (comentário de linha):**
- Entrada: `nome = aluna' --` e `senha = qualquer coisa`
- Query resultante:
  ```sql
  SELECT 1 FROM usuarios WHERE nome = 'aluna' --' AND senha = 'qualquer coisa'
  ```
- O `--` comentar a restante da instrução, e apenas a verificação do nome resta. **Acesso autorizado sem senha.**

Cenário 2 — expressão OR sempre-verdadeira:
- Entrada: `nome = ' OR 1=1 --` e `senha = x`
- Query resultante:
  ```sql
  SELECT 1 FROM usuarios WHERE nome = '' OR 1=1 --' AND senha = 'x'
  ```
- `1=1` é sempre verdadeiro, então retorna o usuário (o primeiro da tabela, ou o único se houver).

Cenário 3 — erro de injeção para mapear a estrutura (se o erro estivesse visível):
- `nome = ' OR senha LIKE 's%'` para listar usuários cujo prefixo da senha seja `s`.

Em resumo: o atacante manipula a **linguagem SQL** dando uma entrada que quebra o contexto de string e altera a lógica da consulta.

Como reescrever com consulta parametrizada

O dado nunca mais entra no texto da query — o SQLite trata o valor
  ```

como uma constante:

```python
def autenticar(banco, nome, senha):
    consulta = "SELECT 1 FROM usuarios WHERE nome = ? AND senha = ?"
    return banco.execute(consulta, (nome, senha)).fetchone() is not None
```

- `?` é um placeholder; `banco.execute()` associa `(nome, senha)` como parâmetros, escapando corretamente e impedindo que o caractere `'` mude o significado da instrução.
- O mesmo padrão vale para `INSERT` e `UPDATE`: `banco.execute("INSERT INTO usuarios VALUES (?, ?)", (nome, senha))`.

Perceba que `criar_banco()` já usa parâmetros (`"? ?"` com tupla) — só a função de autenticação quebrou o padrão.

Quando o MITRE ATT&CK **T1190** se aplica

T1190 — Exploit Public-Facing Application (sub-tema de Initial Access): o atacante explora uma vulnerabilidade em um aplicativo de acesso público para obter acesso inicial.

Aplica-se nesse lab no cenário em que o formulário de login está **exposto a uma rede** — por exemplo, uma aplicação web que recebe `nome` e `senha` via HTTP (`GET`/`POST`). O atacante, sem autenticação, envia os payloads acima pela rede para o endpoint público e, se a aplicação repassá-los para o banco sem parametrização, obtem acesso não autorizado.

Se o `lab/app.py` for executado **apenas em terminal, sem rede** (como o próprio lab descreve), o risco é de **intossicação local** (CWE-89 dentro do processo do usuário), não T1190 — pois não há um aplicativo de acesso público sobre a rede a ser explorado. T1190 só é pertinente quando há exposição de rede.

Outros pontos que notei (além da linha 17)

- **Uso de senha em texto plano:** `senha-lab` armazenada e comparada diretamente. Em produção, seria necessário hashing (ex.: `bcrypt`/`argon2`) e verificação com `checkpw`.
- **Sem registro de tentativas:** não há audit trail de login (um padrão reforçado por segurança).
- **Usuário fixo em texto plano:** `("aluna", "senha-lab")` no `criar_banco` — ok para lab, mas reflete a mesma má prática do `SELECT` com texto plano.

Conclusão: a linha 17 é o único ponto crítico; a correção é a query parametrizada. O T1190 entra se o flow ficar exposto via rede.

Reasoning
